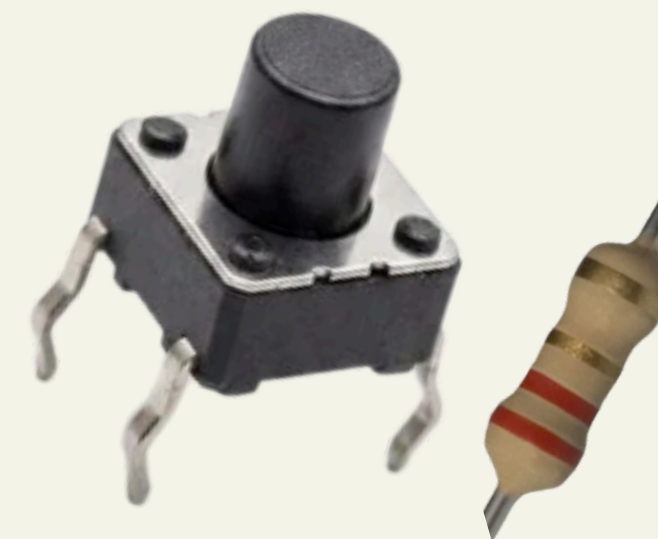


ARDUINO: CONCEITOS, PROGRAMAÇÃO E APLICAÇÃO PRÁTICA



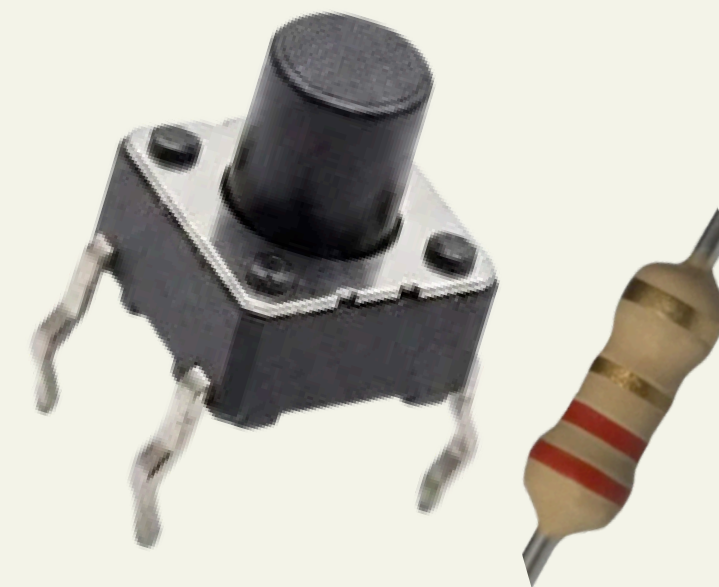
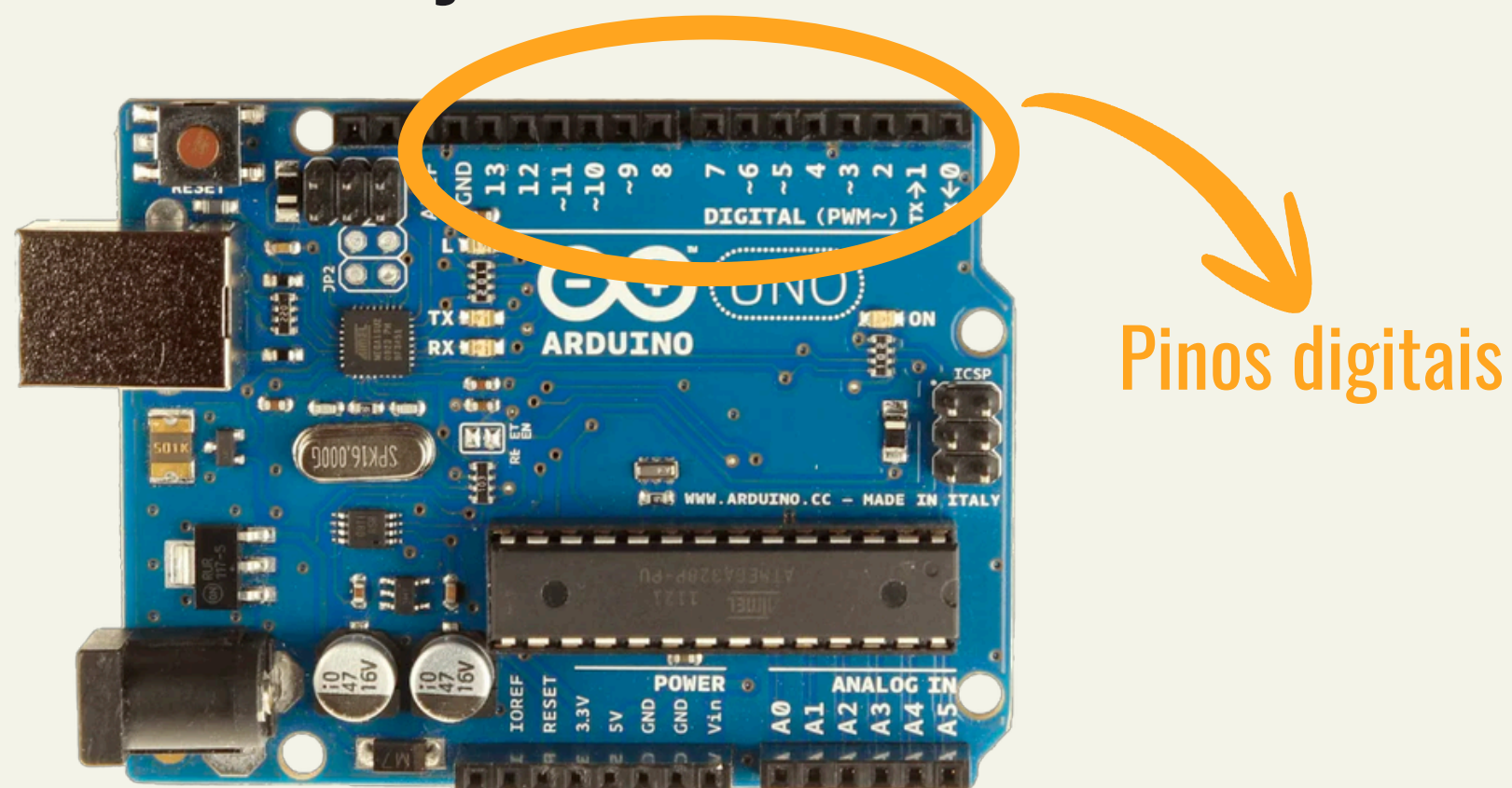
AULA 02

CIRCUITOS ANALÓGICOS, DIGITAIS E ESTRUTURAS DE DECISÃO E REPETIÇÃO



PUSH-BUTTON

- Dispositivo momentâneo que fecha ou abre um circuito quando pressionado.
- Amplamente utilizado para entrada de usuário e controle de eventos.
- No Arduino: Conectado a um pino digital para ler se está pressionado ou não.
 - O pino não pode ficar "solto" (flutuante) quando o botão não está pressionado.
 - Solução: Usar resistor pull-up ou pull-down.



CONFIGURAÇÕES DO PUSH-BUTTON

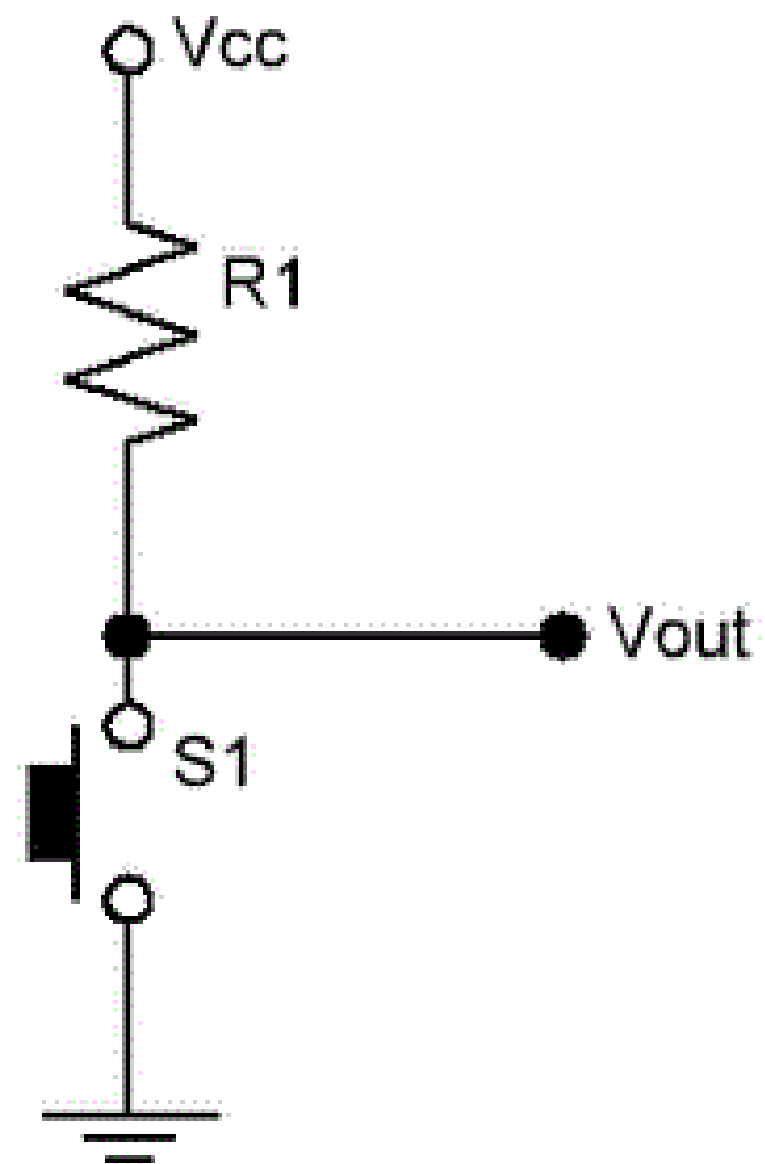
- **Pull-Up:**

- **Pino conectado a uma resistência de pull-up para VCC.**
- **Pino lê HIGH (1) quando não pressionado; muda para LOW (0) quando pressionado.**

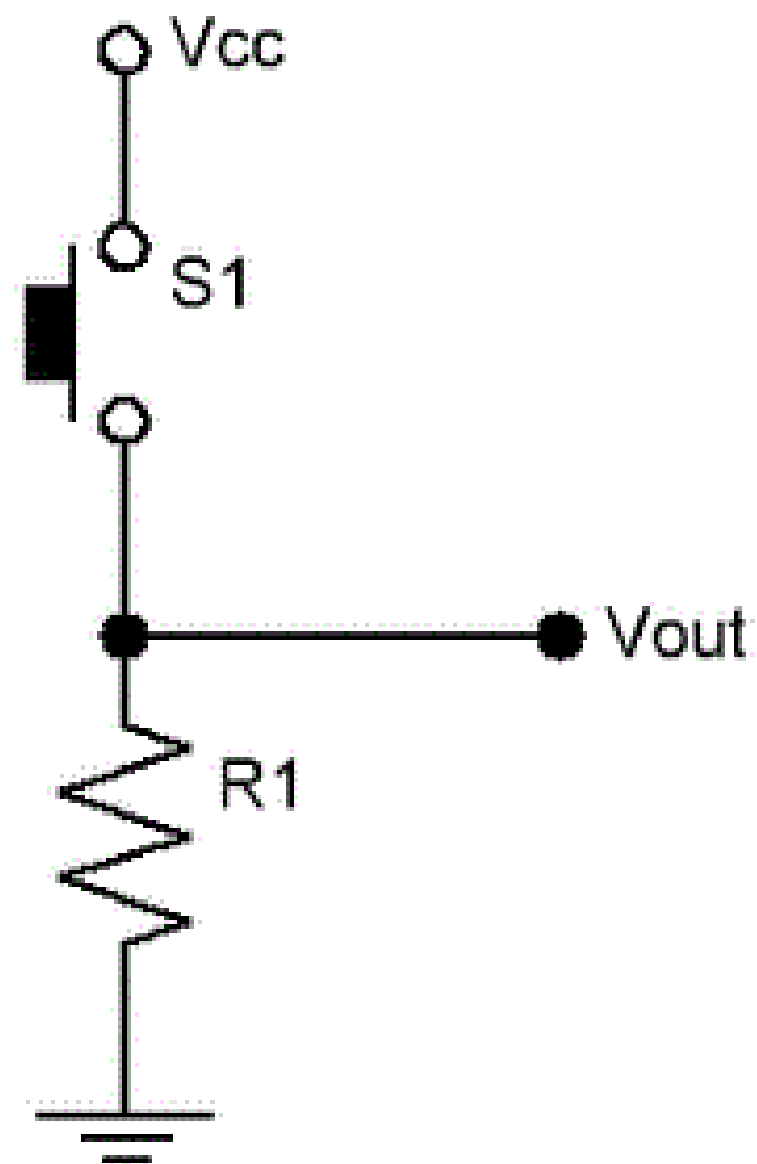
- **Pull-Down:**

- **Resistor externo entre o pino digital e o GND.**
- **Pino lê LOW (0) quando não pressionado; muda para HIGH (1) quando pressionado.**

CONFIGURAÇÕES DO PUSH-BUTTON



Pull Up



Pull Down

IMPORTÂNCIA DA CONFIGURAÇÃO ADEQUADA

- **Push-Button configurado corretamente:**
 - **Leitura estável -> HIGH ou LOW confiáveis.**
- **Push-Button configurado incorretamente:**
 - **Leitura instável -> ativações aleatórias, bugs imprevisíveis.**

CONFIGURAÇÃO INPUT_PULLUP

- **Usa um resistor interno do Arduino para conectar o pino digital ao 5V (HIGH) quando o botão não está pressionado.**
 - **Utiliza menos componentes, já que não precisa de resistor externo.**
 - **Quando o botão está sendo pressionado, o Arduino lê o estado como LOW (0), e quando solto, lê como HIGH (1).**

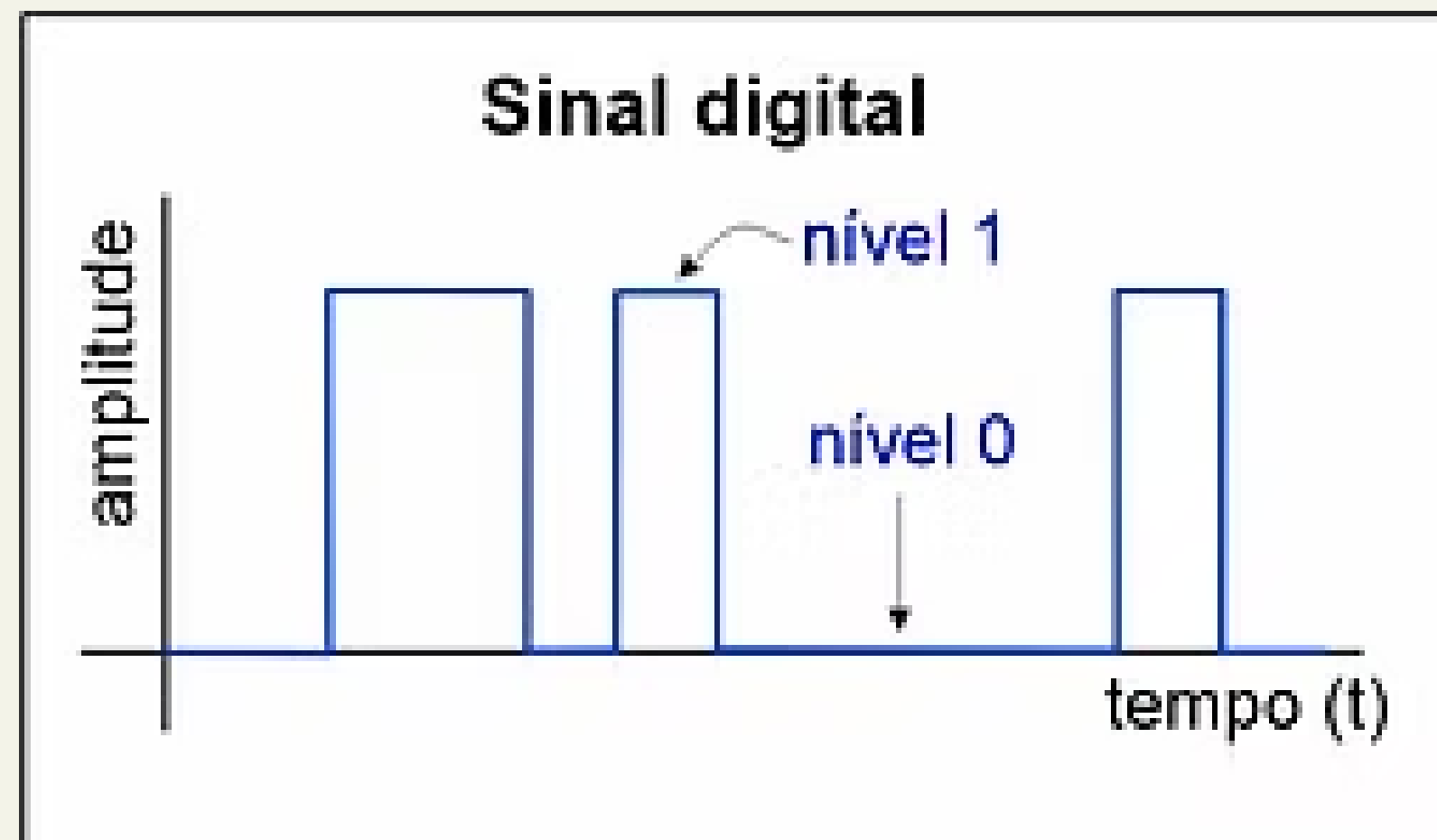
COMPONENTES DIGITAIS X ANALÓGICOS

- **Analógico = Contínuo (Infinitos Valores)**
 - Representa o mundo real de forma suave e gradual.
 - Exemplo: Girar um potenciômetro (a tensão varia continuamente de 0V a 5V).
 - Aplicações típicas: Sensores de luz, temperatura, pressão, amplificadores de áudio.
- **Digital = Discreto (Valores Fixos)**
 - Representa informação em estados bem definidos.
 - Exemplo: Push-Button (só pode estar ligado ou desligado).
 - Aplicações Típicas: Lógica de computadores, controle de dispositivos digitais, comunicação digital.

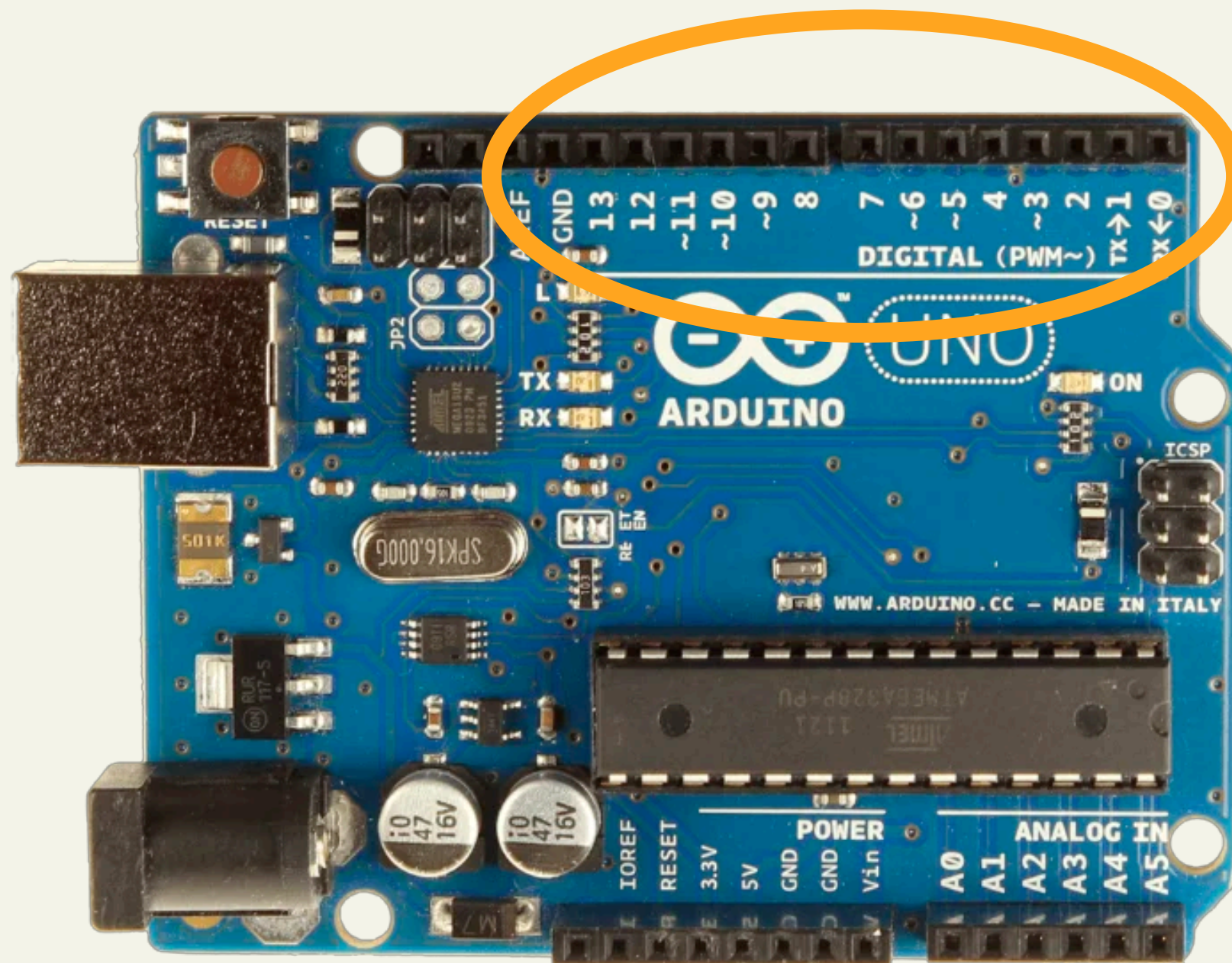
SINAIS DIGITAIS

- **Sinais com apenas dois estados:**
 - **0 (LOW) -> 0V**
 - **1 (HIGH) -> 5V**
- **No Arduino:**
 - **digitalWrite(pino, HIGH/LOW):** Envia sinal digital.
 - **digitalRead(pino):** Lê sinal digital (retorna HIGH ou LOW).
- **Características Importantes:**
 - **Robustez:** Resistente a interferências (só importa se é 0 ou 1).
 - **Simples:** Base de toda a lógica de programação.

SINAIS DIGITAIS



PINOS DIGITAIS NO ARDUINO



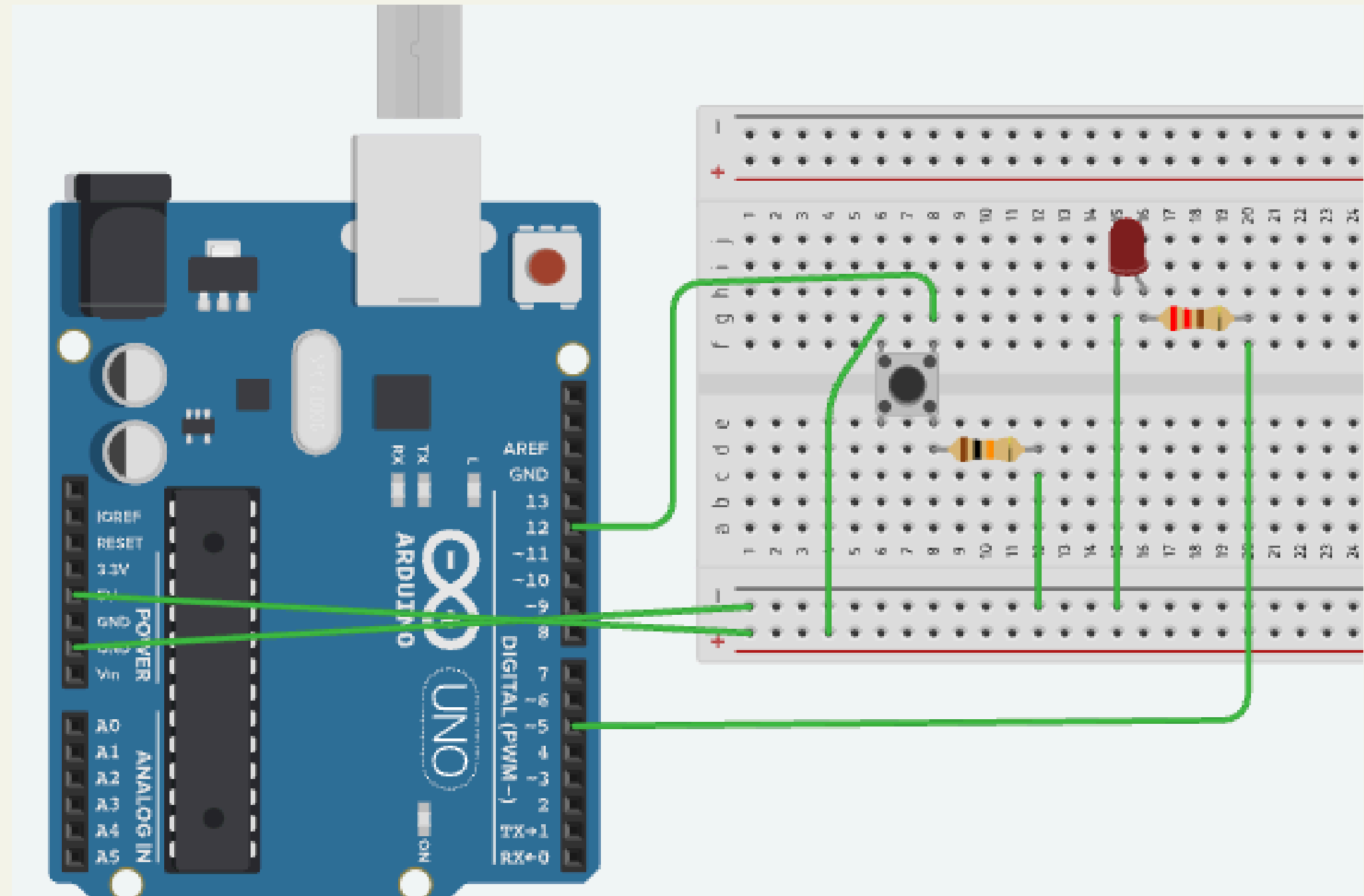
Pinos digitais:
0 a 13

LABORATÓRIO 1: CONTROLE DE LED COM BOTÃO

- **Utilizar um botão (Push-button) para controlar um led, enquanto estiver apertado o led deve acender e quando solto apagar. Para evitar leituras erradas, utilize um resistor de pull-down.**

LABORATÓRIO 1: CONTROLE DE LED COM BOTÃO

- Utilizar um botão (Push-button) para controlar um led, enquanto estiver apertado o led deve acender e quando solto apagar. Para evitar leituras erradas, utilize um resistor de pull-down.



LABORATÓRIO 1: CONTROLE DE LED COM BOTÃO

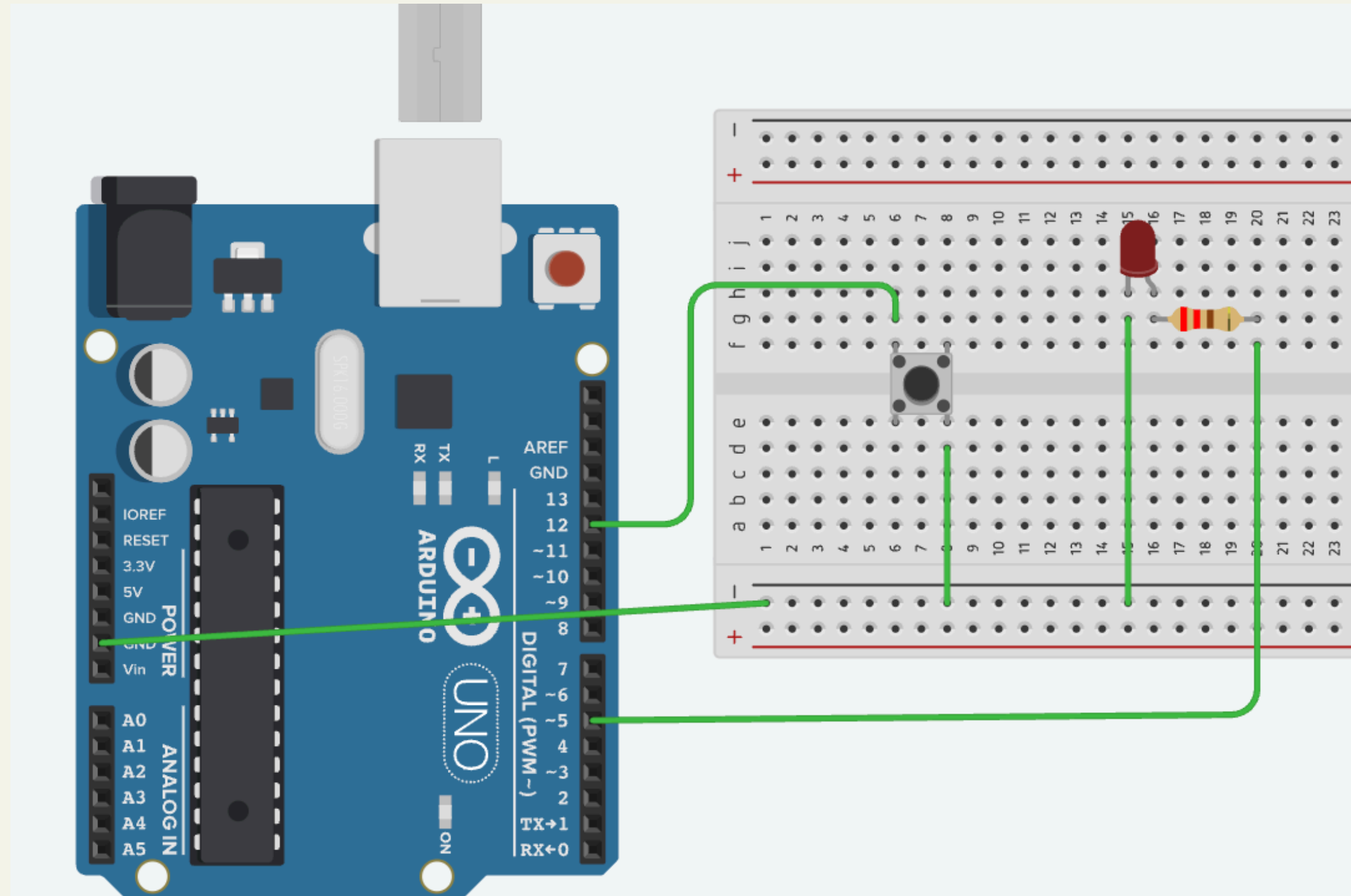
- Utilizar um botão (Push-button) para controlar um led, enquanto estiver apertado o led deve acender e quando solto apagar. Para evitar leituras erradas, utilize um resistor de pull-down.

```
1 // Definição dos pinos
2 int ledPin = 5;    // Pino do LED
3 int botaoPin = 12; // Pino do botão
4
5 void setup() {
6     pinMode(ledPin, OUTPUT);
7     pinMode(botaoPin, INPUT); // Usando resistor pull-down externo
8 }
9
10 void loop() {
11     // Lê o botão e escreve no LED:
12     digitalWrite(ledPin, digitalRead(botaoPin));
13
14     delay(50);
15 }
```

LABORATÓRIO 2: CONTROLE DE LED COM BOTÃO

- **Utilizar um botão (Push-button) para controlar um led, enquanto estiver apertado o led deve acender e quando solto apagar. Para evitar leituras erradas, utilize um resistor de pull-up interno.**

LABORATÓRIO 2: CONTROLE DE LED COM BOTÃO



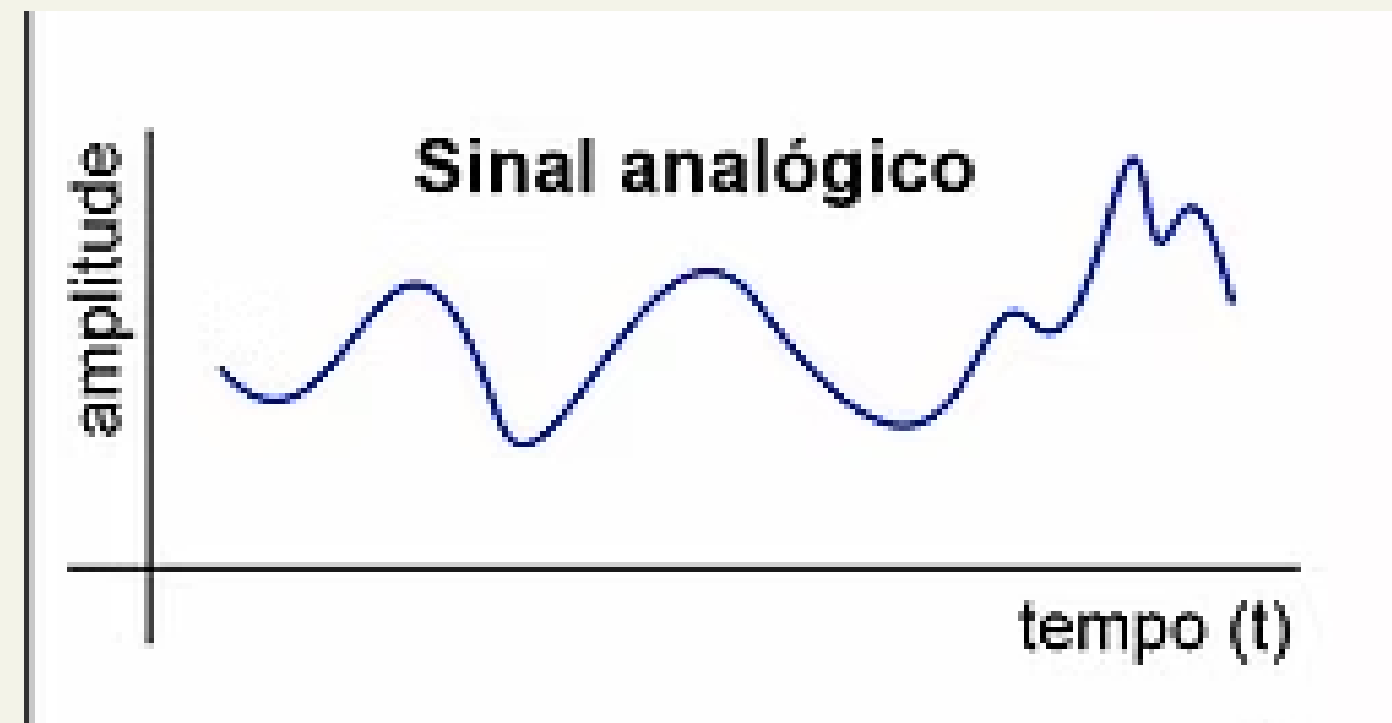
LABORATÓRIO 2: CONTROLE DE LED COM BOTÃO

```
1 // Definição dos pinos
2 int ledPin = 5;    // Pino do LED
3 int botaoPin = 12; // Pino do botão
4
5 void setup() {
6     pinMode(ledPin, OUTPUT);
7     pinMode(botaoPin, INPUT_PULLUP); // Usando resistor pull-down externo
8 }
9
10 void loop() {
11     // Lê o botão e escreve no LED:
12     digitalWrite(ledPin, digitalRead(botaoPin));
13
14     delay(50);
15 }
16
```

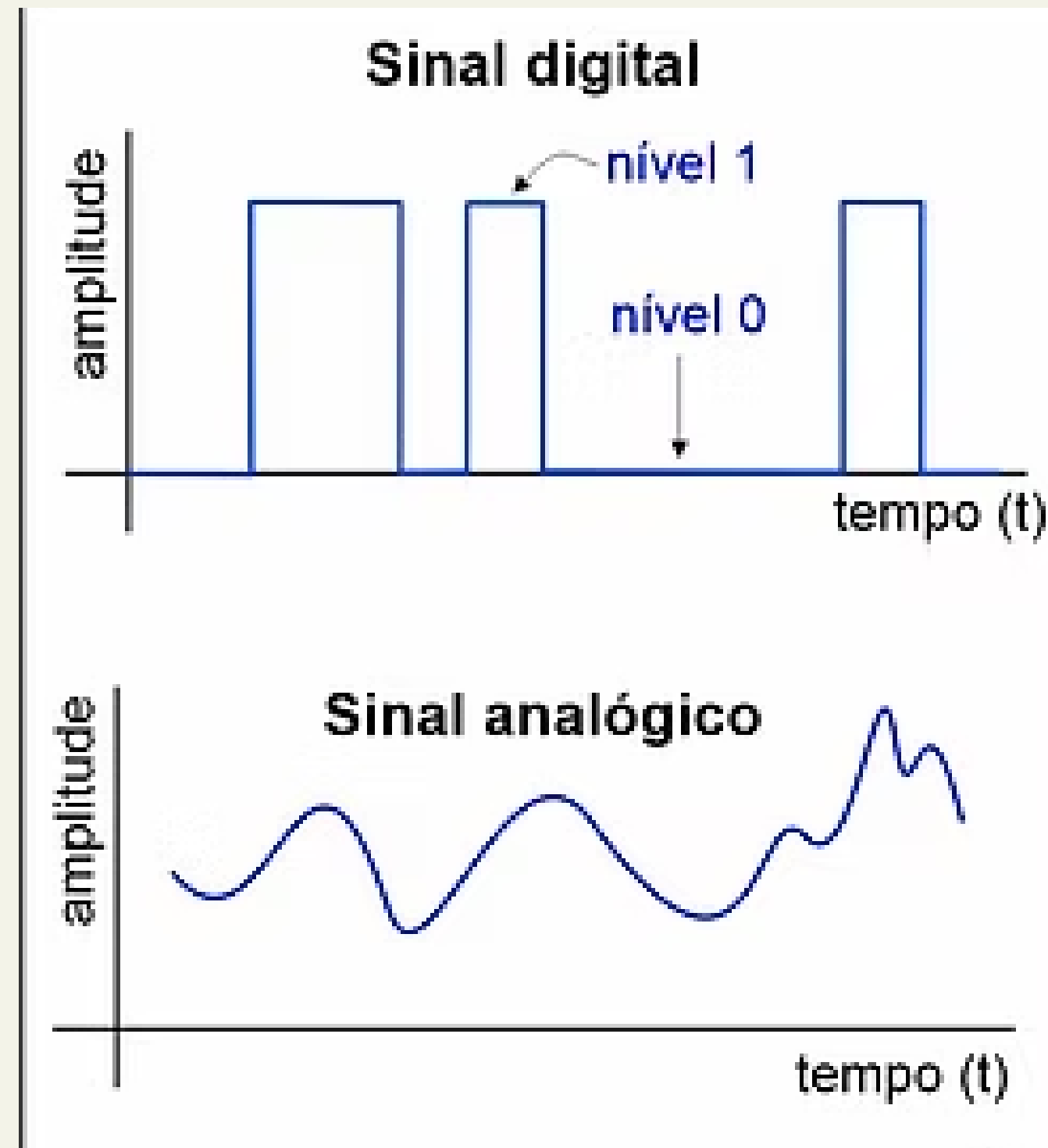
SINAIS ANALÓGICOS

- **Sinais que variam suavemente entre valores infinitos, representando grandezas físicas do mundo real.**
- **Sensores Analógicos Comuns:**
 - **Potenciômetro (controle de volume/brilho)**
 - **Sensor de temperatura (LM35)**
 - **Sensor de luz (LDR)**
 - **Sensor de umidade do solo**

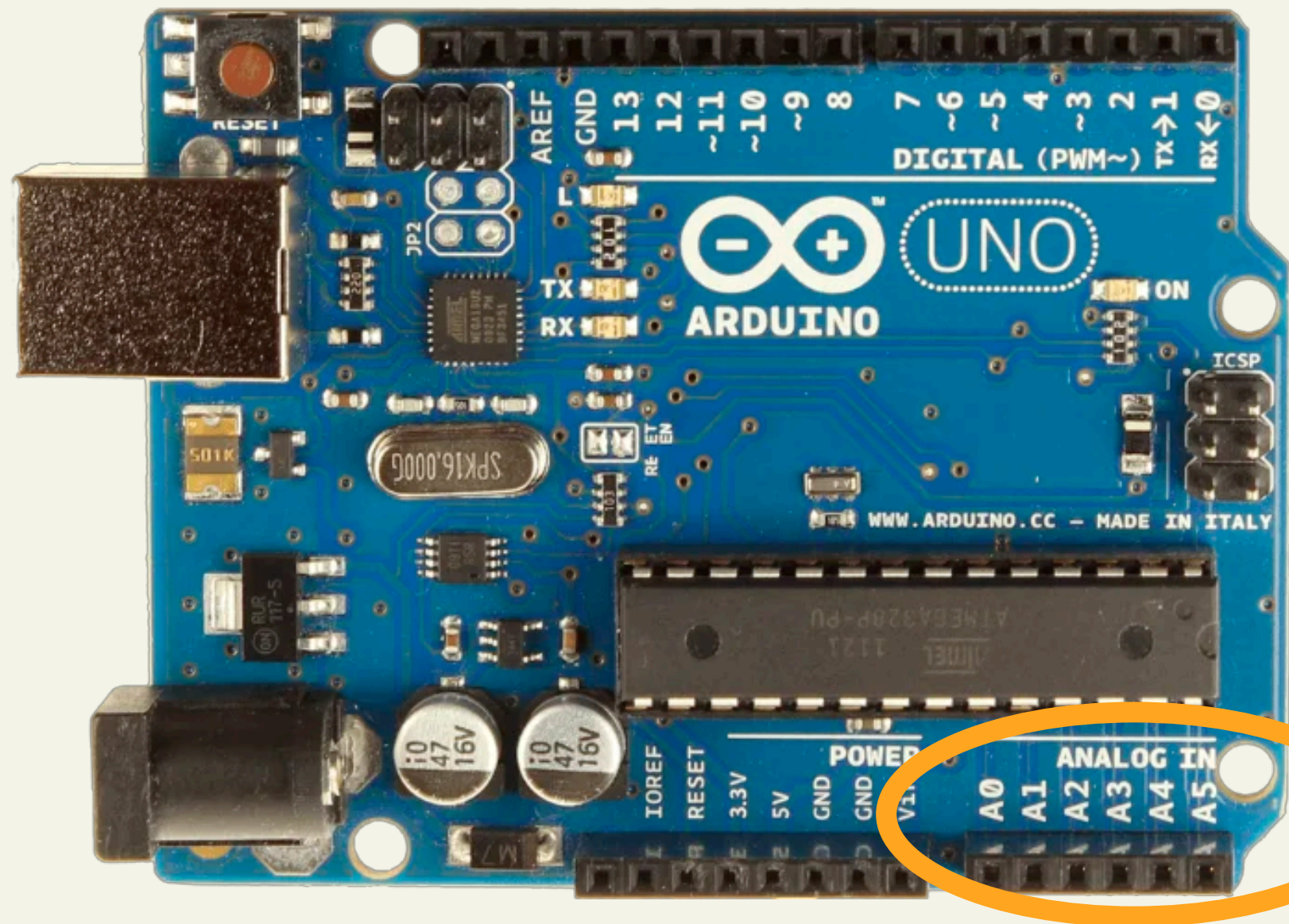
SINAIS ANALÓGICOS



SINAIS ANALÓGICOS



PINOS ANALÓGICOS NO ARDUINO



Pinos analógicos:
A0 a A5

CONVERSÃO ANALÓGICO-DIGITAL

- **O Arduino é digital, mas o mundo é analógico.**
- **Solução: Utilizar o conversor A/D, que transforma tensão contínua em números digitais.**
- **Cada pino analógico (A0-A5) possui um conversor A/D interno.**
 - **Resolução: 10 bits (Converte tensão contínua em $2^{10} = 1024$ degraus digitais)**
- **No código:**
 - **`analogRead(pino)` -> Lê valores de 0 a 1023.**

EXEMPLOS DE FUNCIONAMENTO

- Sensor fornece 2,5 V ao pino A0

5V → 1023

2,5 V → X

5V → 1023

2,5 V → X



$$5 * X = 2,5 * 1023$$

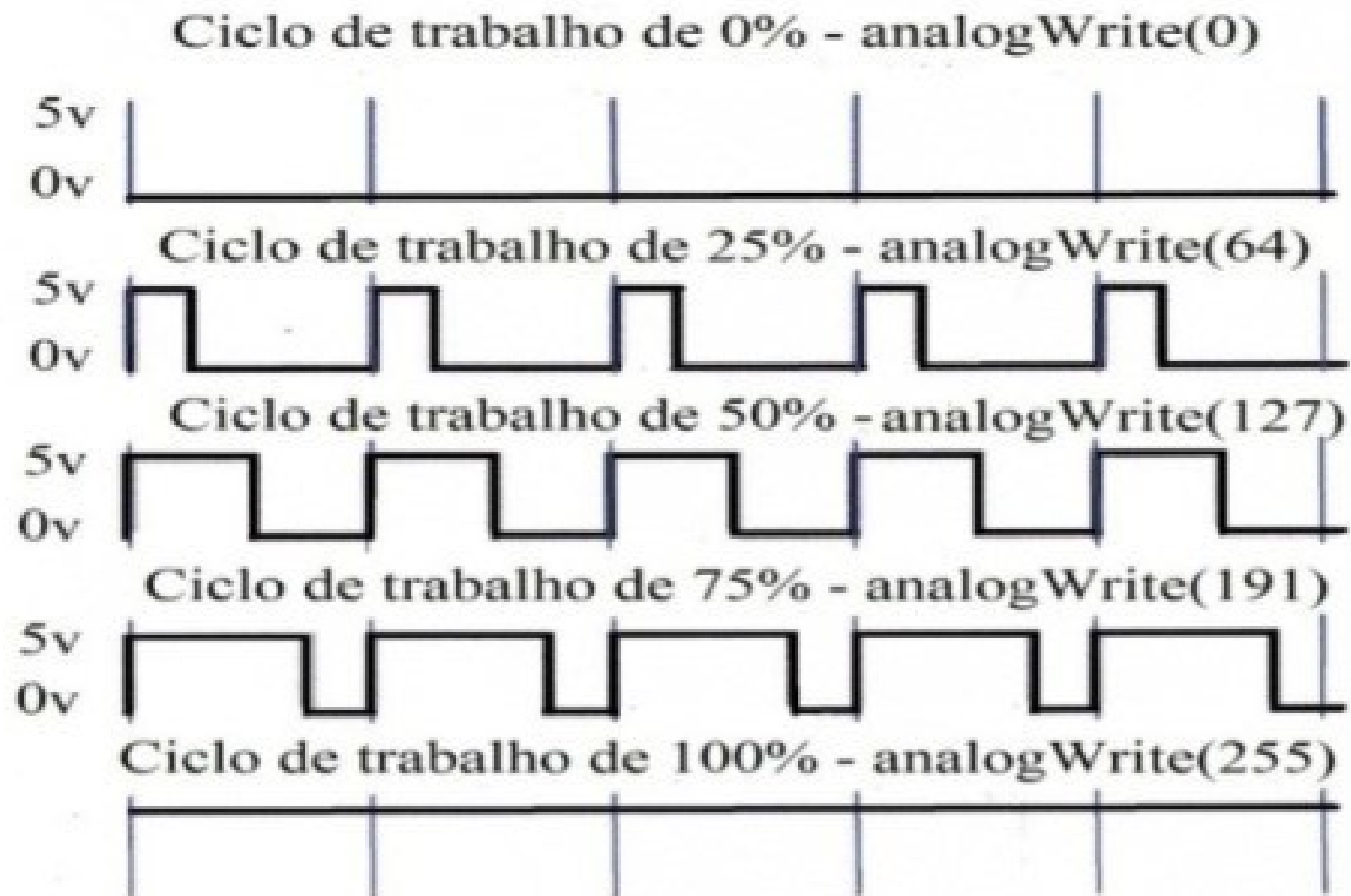
$$X = (2,5 * 1023) / 5$$

$$X \approx 512$$

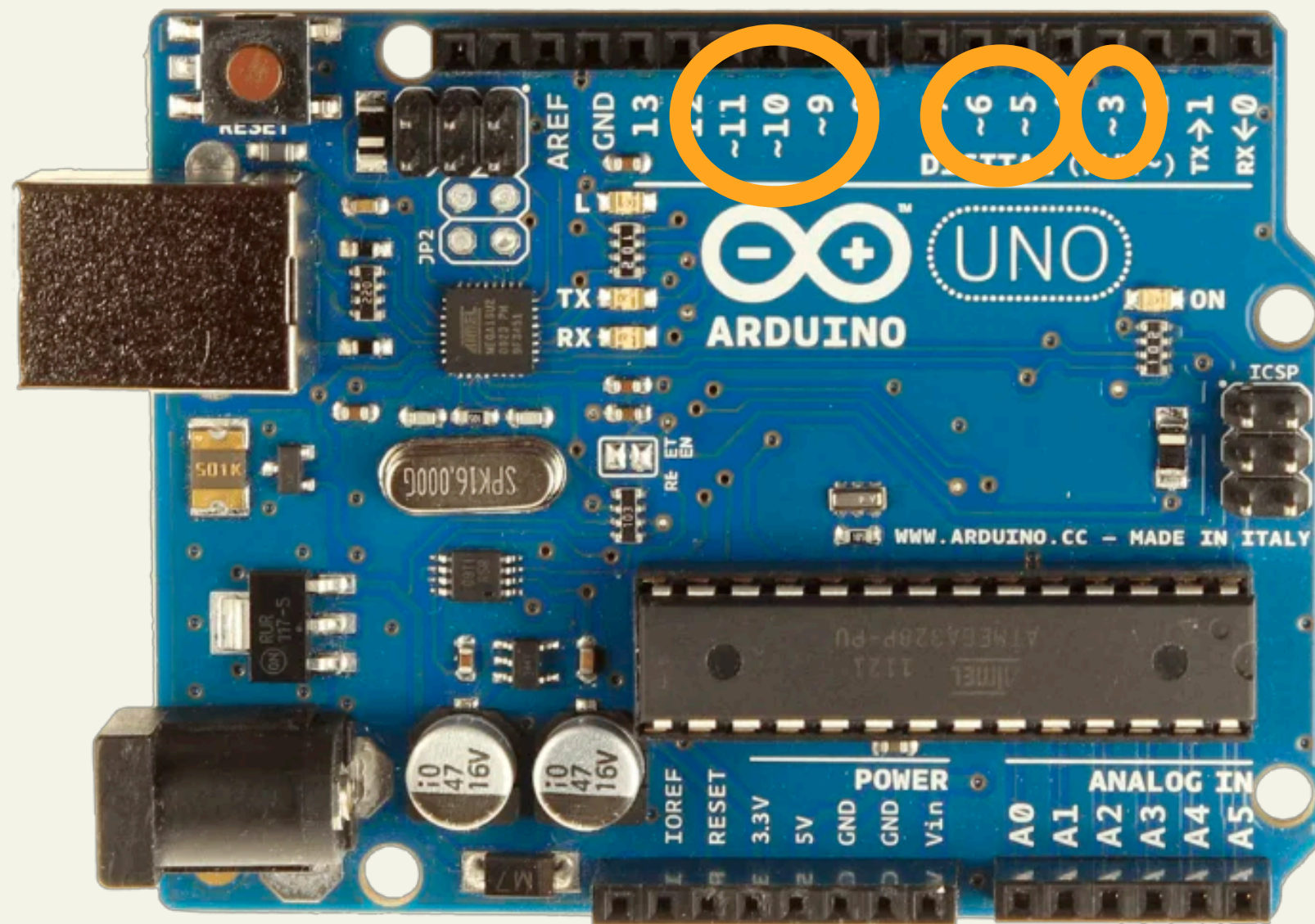
MODULAÇÃO POR LARGURA DE PULSO (PWM)

- **Técnica que simula saída analógica usando pulsos digitais de largura variável, alternando rapidamente entre o estado ligado (HIGH) e desligado (LOW).**
- **Características do PWM:**
 - **Frequência (pulsos por segundo).**
 - **Ciclo de trabalho (Percentual do tempo em HIGH)**
- **No Arduino: Função `analogWrite(pino, valor)`, com valor indo de 0 (0% da potência) a 255 (100% da potência)**
- **Exemplo de Aplicações:**
 - **Controle de luminosidade em LEDs**
 - **Controle de velocidade em motores**

MODULAÇÃO POR LARGURA DE PULSO (PWM)



PINOS PWM NO ARDUINO



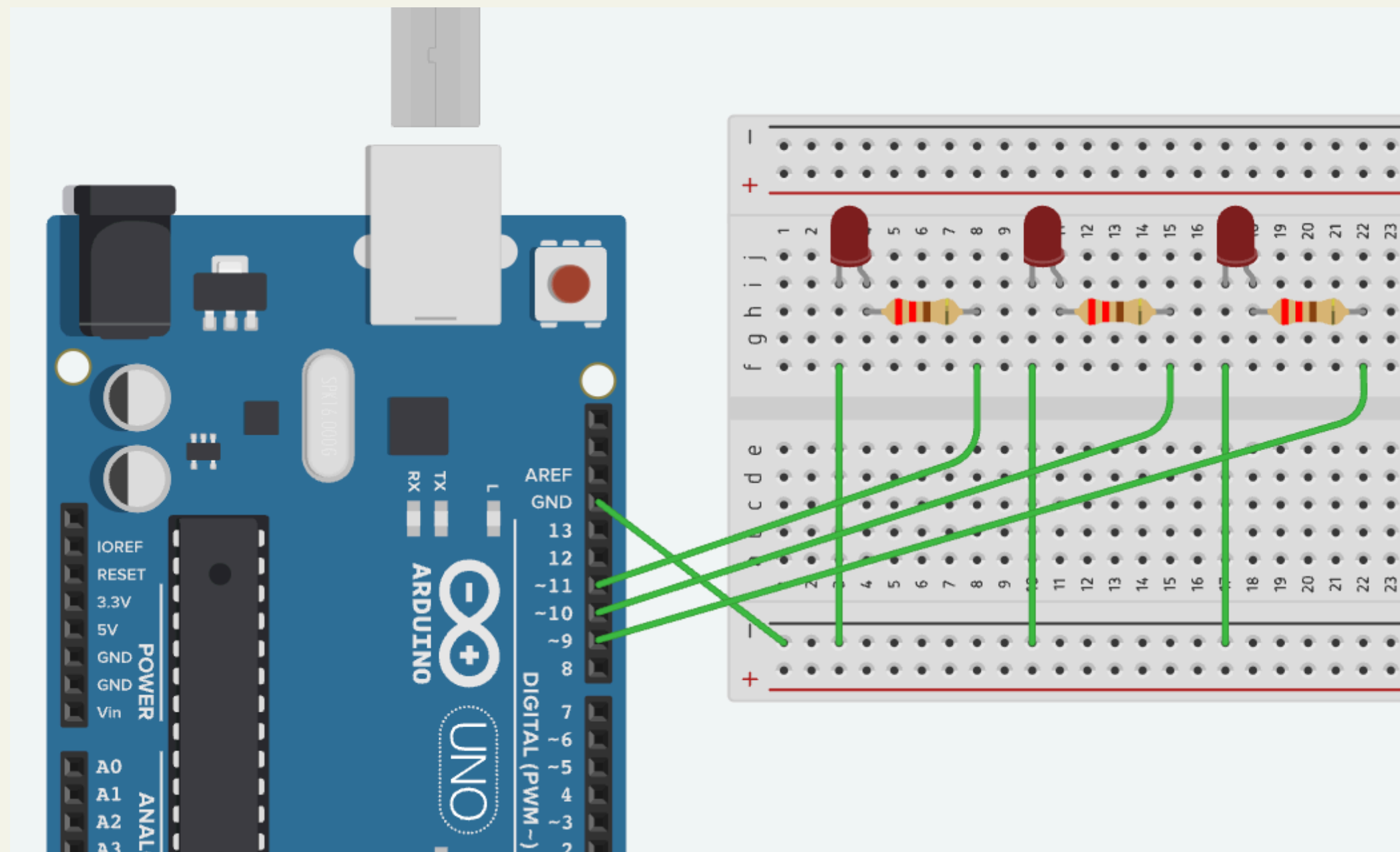
Pinos PWM:
3, 5, 6, 9, 10 e 11

LABORATÓRIO 3: VISUALIZANDO A AÇÃO DO PWM COM LEDs

- **Configure 3 pinos, cada um para controlar um LED. Utilize sinais PWM para controlar a intensidade de cada um dos LEDs.**

LABORATÓRIO 3: VISUALIZANDO A AÇÃO DO PWM COM LEDs

- Configure 3 pinos, cada um para controlar um LED. Utilize sinais PWM para controlar a intensidade de cada um dos LEDs.



LABORATÓRIO 3: VISUALIZANDO A AÇÃO DO PWM COM LEDs

```
1 // Definição dos pinos dos LEDs
2 int led1 = ____; // Pino PWM
3 int led2 = ____; // Pino PWM
4 int led3 = ____; // Pino PWM
5
6 void setup() {
7     // Configura os pinos como saída
8     pinMode(led1, OUTPUT);
9     pinMode(____, OUTPUT);
10    pinMode(led3, ____);
11 }
12
13 void loop() {
14     // LED 1 com brilho baixo (25% = 64/255)
15     _____(led1, ____);
16
17     // LED 2 com brilho médio (50% = 128/255)
18     _____(____, 128);
19
20     // LED 3 com brilho alto (75% = 192/255)
21     _____(led3, ____);
22 }
23
```

LABORATÓRIO 3: VISUALIZANDO A AÇÃO DO PWM COM LEDs

```
1 // Definição dos pinos dos LEDs
2 int led1 = 9; // Pino PWM
3 int led2 = 10; // Pino PWM
4 int led3 = 11; // Pino PWM
5
6 void setup() {
7     // Configura os pinos como saída
8     pinMode(led1, OUTPUT);
9     pinMode(led2, OUTPUT);
10    pinMode(led3, OUTPUT);
11 }
12
13 void loop() {
14     // LED 1 com brilho baixo (25% = 64/255)
15     analogWrite(led1, 64);
16
17     // LED 2 com brilho médio (50% = 128/255)
18     analogWrite(led2, 128);
19
20     // LED 3 com brilho alto (75% = 192/255)
21     analogWrite(led3, 192);
22 }
23
```

OPERADORES

- **Símbolos que realizam operações com valores e variáveis. Permitem que o programa realize cálculos, comparações e tome decisões. Eles podem ser classificados em diferentes grupos, sendo os principais:**
 - **Aritméticos: Fazem contas**
 - **Atribuição: Guardam resultados**
 - **Comparação: Fazem perguntas (sim/não)**
 - **Lógicos: Combinam condições**
 - **Incremento/Decremento: Somam/subtraem 1**

OPERADORES ARITMÉTICOS

- **'+' (adição):**
 - **Exemplo: `int resultado = a + b;`**
- **'-' (subtração):**
 - **Exemplo: `int resultado = a - b;`**
- **'*' (multiplicação):**
 - **Exemplo: `int resultado = a * b;`**
- **'/' (divisão):**
 - **Exemplo: `int resultado = a / b;`**
- **'%' (módulo):**
 - **Exemplo: `int resultado = a % b;`**

OPERADORES DE ATRIBUIÇÃO

- **'=' (atribuição):**
 - **Exemplo: `int a = 5;`**
- **'+=' (adição e atribuição):**
 - **Exemplo: `a += 3;`**
 - **// Equivalente a `a = a + 3;`**
- **'-=' (subtração e atribuição):**
 - **Exemplo: `a -= 2;`**
 - **// Equivalente a `a = a - 2;`**
- **É válido para todos os outros operadores aritméticos.**

OPERADORES DE COMPARAÇÃO:

- **'==' (igual a):**
 - **Exemplo: if (a == b) { /* código */ }**
- **'!=' (diferente de):**
 - **Exemplo: if (a != b) { /* código */ }**
- **'<', '>', '<=', '>=' (menor que, maior que, menor ou igual a, maior ou igual a):**
 - **Exemplo: if (a > b) { /* código */ }**

OPERADORES LÓGICOS E INCREMENTO/DECREMENTO

- **'&&' (E lógico):**
 - Exemplo: `if (condicao1 && condicao2) { /* código */ }`
- **'||' (OU lógico):**
 - Exemplo: `if (condicao1 || condicao2) { /* código */ }`
- **'!' (NÃO lógico):**
 - Exemplo: `if (!condicao) { /* código */ }`
- **'++' (incremento) e '--' (decremento):**
 - Exemplo: `a++;` (incremento de 'a'; `a = a+1`)

ESTRUTURAS DE CONTROLE DE DECISÃO

- **Blocos que permitem tomar decisões no código, executando ações diferentes dependendo de condições.**
- **Estruturas Comuns no Arduino:**
 - **"if, else if, else":**
 - **Verificações condicionais simples ou encadeadas.**
 - **Executa bloco de código se a condição for verdadeira.**
 - **"switch, case":**
 - **Adequado para múltiplas opções baseadas na mesma variável.**
 - **Executa bloco de código se a opção for selecionada.**

EXEMPLO: IF/ELSE

```
int temperatura = 25;

if (temperatura < 15) {
    digitalWrite(8, HIGH);
}
else if (temperatura < 30) {
    digitalWrite(9, HIGH);
}
else {
    digitalWrite(10, HIGH);
}
```

EXEMPLO: SWITCH CASE

```
int modo = 2

switch (modo) {
  case 1:
    digitalWrite(ledVermelho, HIGH);
    break;

  case 2:
    digitalWrite(ledVerde, HIGH);
    break;

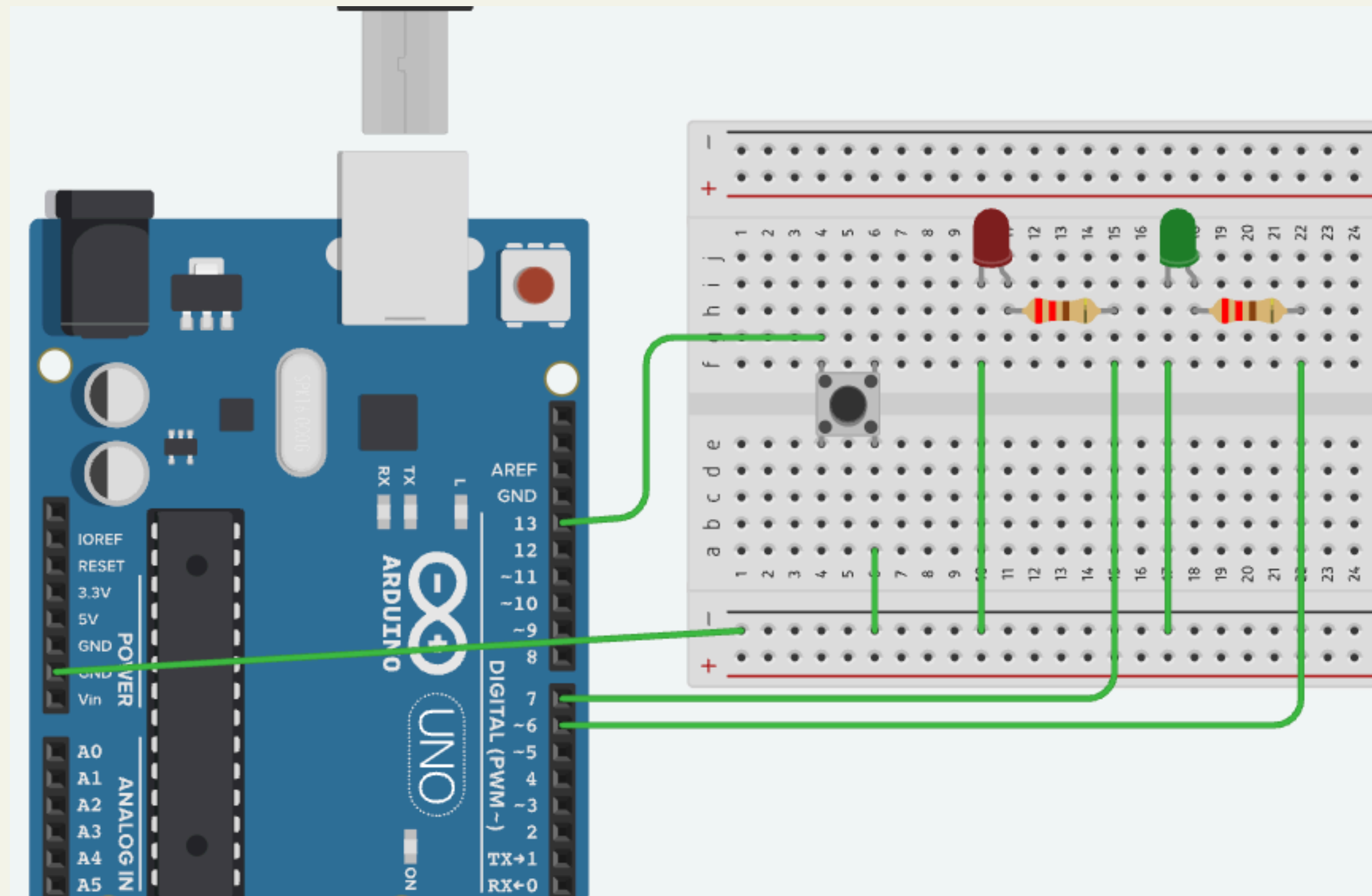
  case 3:
    digitalWrite(ledAzul, HIGH);
    break;

  default:
    // Nenhum LED aceso
    break;
}
```

LABORATÓRIO 4: ALTERNANDO LEDS COM BOTÃO

- **Utilize um botão para determinar qual LED será aceso. Enquanto o botão estiver sendo apertado, um LED verde deverá acender, e quando solto, um LED vermelho deverá acender**

LABORATÓRIO 4: ALTERNANDO LEDS COM BOTÃO



LABORATÓRIO 4: ALTERNANDO LEDS COM BOTÃO

```
1 // Definição dos pinos
2 int ledVermelho = ____; // Pino do LED vermelho
3 int ledVerde = ____; // Pino do LED verde
4 int botaoPin = ____; // Pino do botão
5
6 void setup() {
7     // Configura os LEDs como saída
8     pinMode(ledVermelho, ____);
9     pinMode(ledVerde, ____);
10
11     // Configura o botão como entrada com resistor pull-up interno
12     pinMode(botaoPin, ____);
13 }
14
15 void loop() {
16     // Lê o estado do botão
17     int estadoBotao = ____ (botaoPin);
18
19     // Se o botão for pressionado (LOW com pull-up)...
20     if (estadoBotao == ____) {
21         // Acende LED VERDE e apaga LED VERMELHO
22         digitalWrite(ledVermelho, ____);
23         digitalWrite(ledVerde, ____);
24     } else {
25         // Botão solto: acende LED VERMELHO e apaga LED VERDE
26         digitalWrite(ledVermelho, ____);
27         digitalWrite(ledVerde, ____);
28     }
29
30     delay(50);
31 }
```

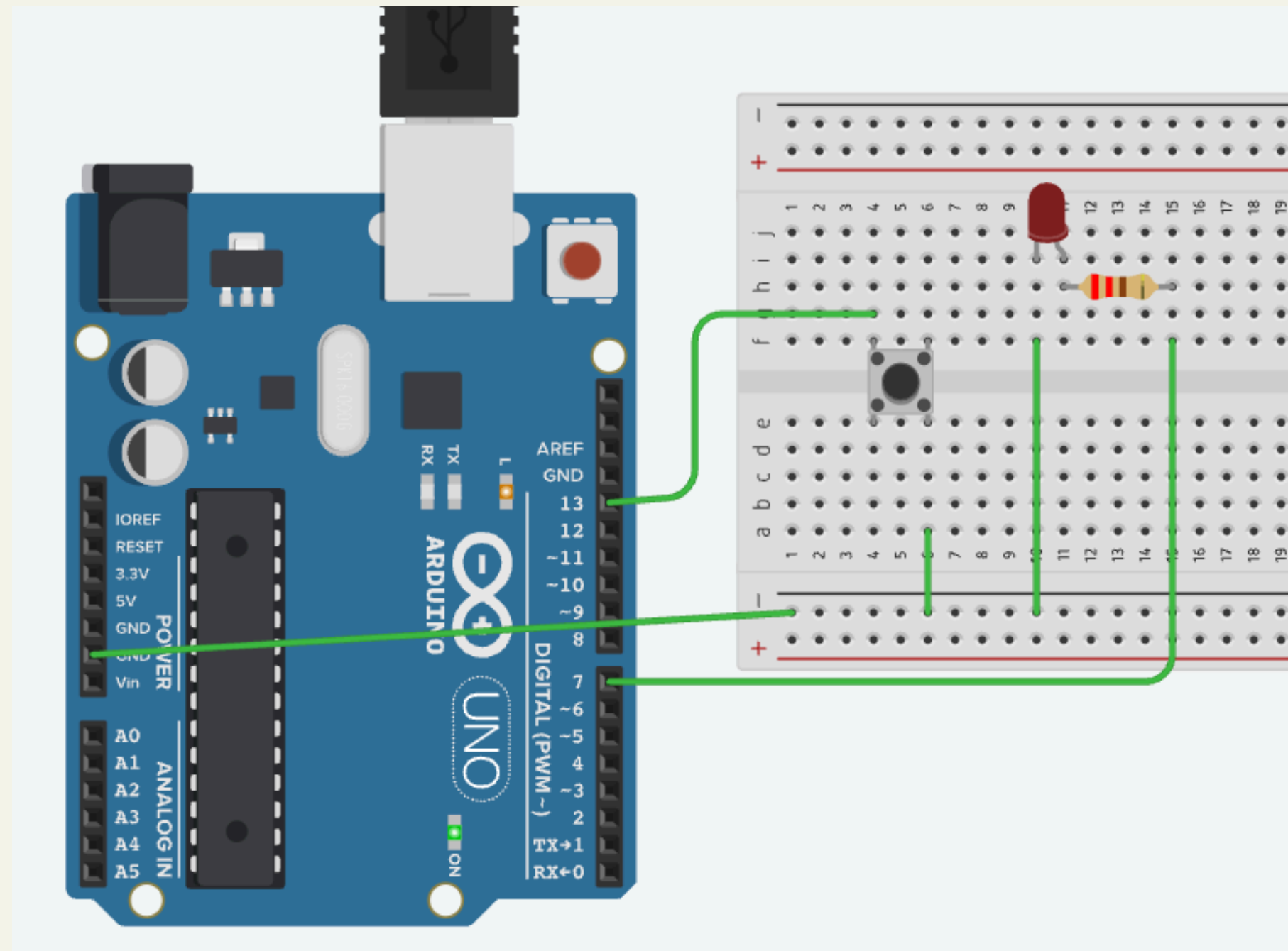
LABORATÓRIO 4: ALTERNANDO LEDs COM BOTÃO

```
1 // Definição dos pinos
2 int ledVermelho = 7; // Pino do LED vermelho
3 int ledVerde = 6; // Pino do LED verde
4 int botaoPin = 13; // Pino do botão
5
6 void setup() {
7     // Configura os LEDs como saída
8     pinMode(ledVermelho, OUTPUT);
9     pinMode(ledVerde, OUTPUT);
10    // Configura o botão como entrada com resistor pull-up interno
11    pinMode(botaoPin, INPUT_PULLUP);
12 }
13
14 void loop() {
15     // Lê o estado do botão
16     int estadoBotao = digitalRead(botaoPin);
17
18     // Se o botão for pressionado (LOW com pull-up)...
19     if (estadoBotao == LOW) {
20         // Acende LED VERDE e apaga LED VERMELHO
21         digitalWrite(ledVermelho, LOW);
22         digitalWrite(ledVerde, HIGH);
23
24     } else {
25         // Botão solto: acende LED VERMELHO e apaga LED VERDE
26         digitalWrite(ledVermelho, HIGH);
27         digitalWrite(ledVerde, LOW);
28     }
29
30     delay(50);
31 }
```

LABORATÓRIO 5: AUMENTANDO BRILHO DO LED COM BOTÃO

- **Utilize um botão para aumentar o brilho de um LED. Caso o valor ultrapasse o valor máximo (255), deverá permanecer em 255.**

LABORATÓRIO 5: AUMENTANDO BRILHO DO LED COM BOTÃO



LABORATÓRIO 5: AUMENTANDO BRILHO DO LED COM BOTÃO

```
1  int luminosidade = ____; // Variável para armazenar o valor do PWM
2  int botao = ____;
3  int led_VERMELHO = ____;
4
5  void setup() {
6      pinMode(botao, ____);
7      pinMode(led_VERMELHO, ____);
8  }
9
10 void loop() {
11     // Se o botão for pressionado...
12     if(____(botao) == ____) {
13         // Aumenta a luminosidade em 1
14         luminosidade = ____ + 1;
15
16         // Se atingir ou passar de 255, mantém em 255
17         if(luminosidade ____ 255) {
18             luminosidade = ____;
19         }
20     }
21
22     // Aplica a intensidade ao LED
23     ____ (led_VERMELHO, luminosidade);
24     delay(____);
25 }
```

LABORATÓRIO 5: AUMENTANDO BRILHO DO LED COM BOTÃO

```
1  int luminosidade = 0; // Variável para armazenar o valor do PWM
2  int botao = 13;
3  int led_VERMELHO = 7;
4
5  void setup() {
6      pinMode(botao, INPUT_PULLUP);
7      pinMode(led_VERMELHO, OUTPUT);
8  }
9
10 void loop() {
11     // Se o botão for pressionado...
12     if(digitalRead(botao) == LOW) {
13         // Aumenta a luminosidade em 1
14         luminosidade = luminosidade + 1;
15
16         // Se atingir ou passar de 255, mantém em 255
17         if(luminosidade >= 255) {
18             luminosidade = 255;
19         }
20     }
21
22     // Aplica a intensidade ao LED
23     analogWrite(led_VERMELHO, luminosidade);
24     delay(50);
25 }
```

ESTRUTURAS DE REPETIÇÃO NO ARDUINO

- Blocos que permitem executar o mesmo código várias vezes, automatizando tarefas repetitivas.
- Estruturas Comuns no Arduino:
 - “for”: Ideal quando se sabe quantas vezes repetir.

```
for (int i = 0; i < 10; i++) {  
    // Código repete 10 vezes.  
}
```

ESTRUTURAS DE REPETIÇÃO NO ARDUINO

- Estruturas Comuns no Arduino:

- “while”: Repete enquanto uma condição for verdadeira.

```
while (digitalRead(botao) == HIGH) {  
    // Fica neste bloco de código até o botão ser pressionado.  
}
```

- “do while”: Executa pelo menos uma vez, depois verifica a condição.

```
do {  
    // Executa primeiro, depois verifica.  
} while (digitalRead(botao) == HIGH);
```

EXEMPLO: ESTRUTURAS DE REPETIÇÃO

```
for (int i = 0; i < 5; i++) {  
    digitalWrite(8, HIGH);  
    delay(500);  
    digitalWrite(8, LOW);  
    delay(500);  
}
```

LABORATÓRIO 6: PISCANDO LED EM NÚMEROS PARES

- **Faça um contador de 0 a 100, e quando o número for par, o LED é acesso, quando impar, o LED é apagado.**

LABORATÓRIO 6: PISCANDO LED EM NÚMEROS PARES

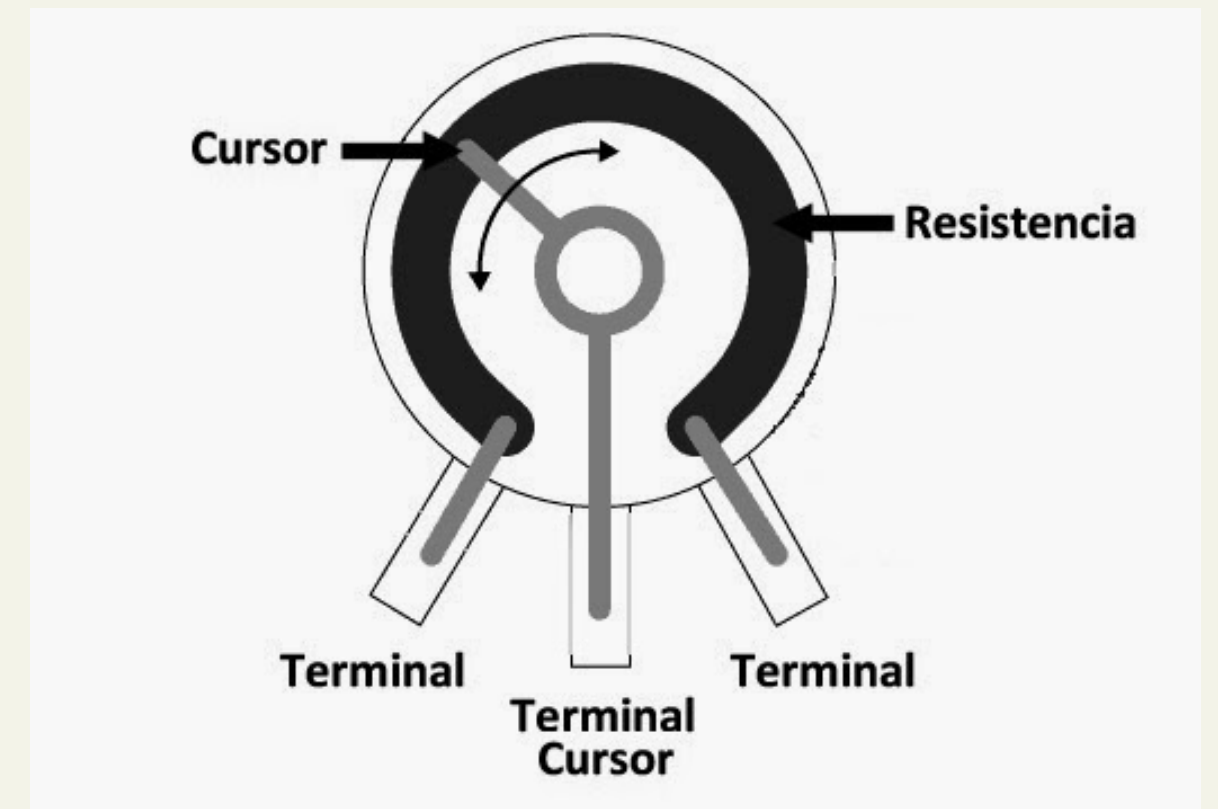
```
1  int LED_VERMELHO = 7;
2
3  void setup()
4  {
5      // Defina o LED como saída:
6      pinMode(LED_VERMELHO, ____);
7  }
8
9  void loop()
10 {
11     // Coloque o contador de 0 a 100:
12     for(int contador = ____; contador < ____; contador++) {
13         if(contador % 2 == 0){
14             // Liga o LED:
15             digitalWrite(LED_VERMELHO, ____);
16             delay(1000);
17         } else{
18             // Apaga o LED:
19             digitalWrite(LED_VERMELHO, ____);
20             delay(1000);
21         }
22     }
23 }
```

POTENCIÔMETRO

- **O potenciômetro é um componente eletrônico com resistência ajustável, controlado por um botão giratório.**
- **Tipos Comuns:**
 - **Rotativo (Single-turn)**
 - **Mais comum para Arduino, usado para controle de volume, brilho, etc.**
 - **Multivolta (Multi-turn)**
 - **Precisão maior (ex: 10 giros completos), usado para instrumentação e ajustes finos.**
 - **Linear**
 - **Controle por cursor deslizante, usado em mixers de áudio, equalizadores, etc.**

FUNCIONAMENTO E CONEXÃO DO POTENCIÔMETRO

- **É um divisor de tensão variável:**
 - Giro total para um lado: Saída = 0V
 - Giro total para outro lado: Saída = 5V
 - Giro pela metade: Saída proporcional (ex: 2.5V)
- **No Arduino:**
 - `analogRead(A0)` converte essa tensão (0-5V) em um número digital (0-1023).



CONEXÃO COM ARDUINO

- Para utilização do potenciômetro, ele deve ser conectado ao arduíno em uma porta analógica, no GND e no VCC



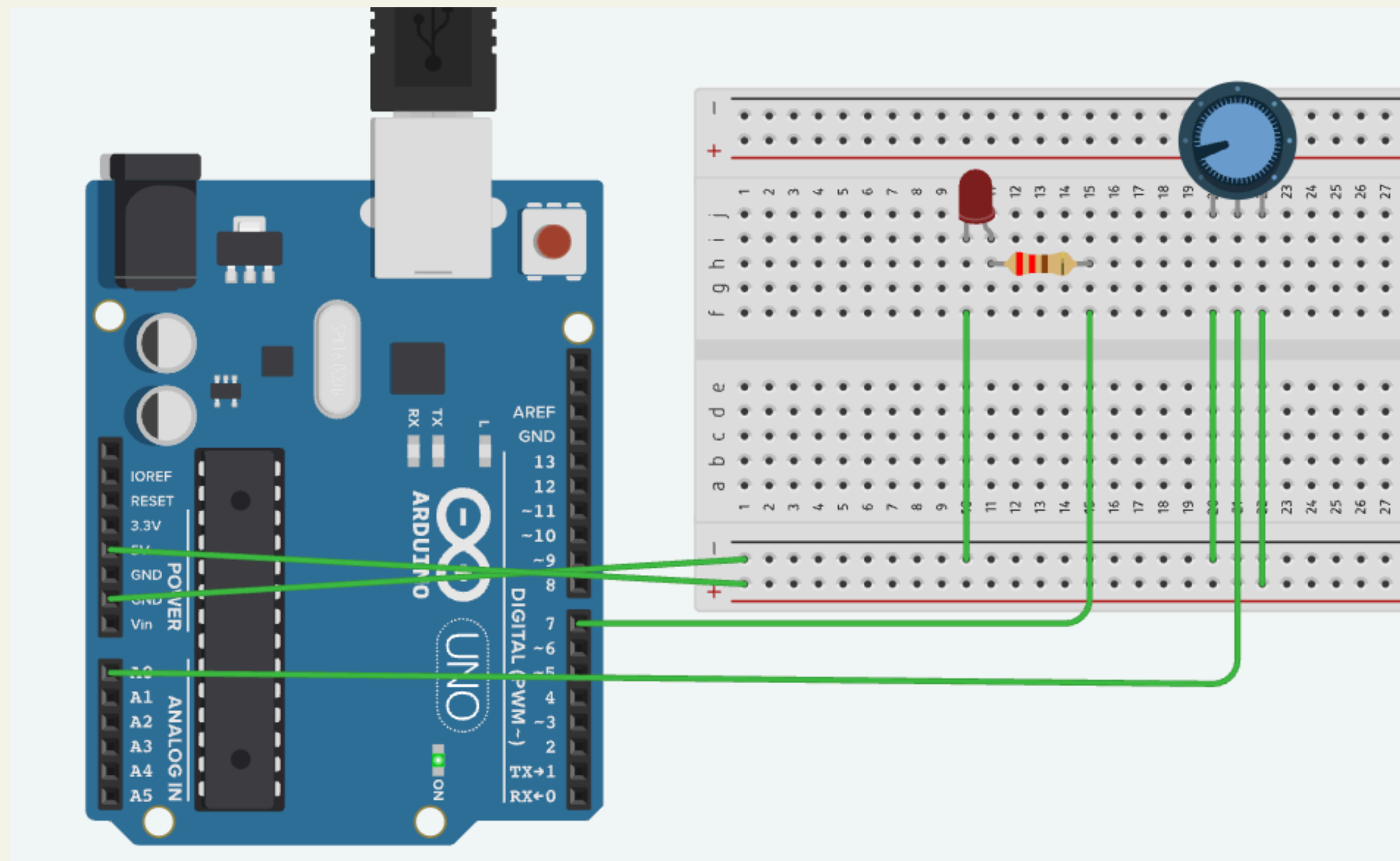
FUNÇÃO MAP()

- **Sintaxe:** `map(valor, deLimiteInferior, deLimiteSuperior, paraLimiteInferior, paraLimiteSuperior)`
- **Parâmetros:**
 - **valor:** O valor a ser mapeado.
 - **deLimiteInferior e deLimiteSuperior:** Limites do intervalo original.
 - **paraLimiteInferior e paraLimiteSuperior:** Limites desejados para o mapeamento.
- **Funcionamento:** Transformação linear ajusta o valor de entrada proporcionalmente entre os intervalos especificados.

LABORATÓRIO 7: CONTROLE DE BRILHO COM POTENCIÔMETRO

- **Utilize um potenciômetro para controlar o brilho de um LED. Para evitar cálculos, pode-se utilizar a função `map()`.**

LABORATÓRIO 7: CONTROLE DE BRILHO COM POTENCIÔMETRO



LABORATÓRIO 7: CONTROLE DE BRILHO COM POTENCIÔMETRO

```
1  const int ledPin = 7;
2  const int potPin = A0;
3
4  void setup() {
5      pinMode(ledPin, OUTPUT);
6  }
7
8  void loop() {
9      int valorPot = analogRead(potPin);
10     int brilho = map(valorPot, 0, 1023, 0, 255);
11     analogWrite(ledPin, brilho);
12
13     delay(100);
14 }
```

BUZZER

- **Componente eletrônico que converte sinais elétricos em ondas sonoras audíveis.**
- **Tipos de Buzzers:**
 - **Passivo:**
 - **Requer sinal PWM para produzir diferentes frequências.**
 - **Composto por diafragma móvel e bobina.**
 - **Ativo:**
 - **Contém circuito oscilador interno para gerar onda sonora.**
 - **Pode ser controlado por sinal lógico para ligar/desligar.**
- **Aplicações Comuns:**
 - **Alarmes, indicadores sonoros, jogos, etc.**

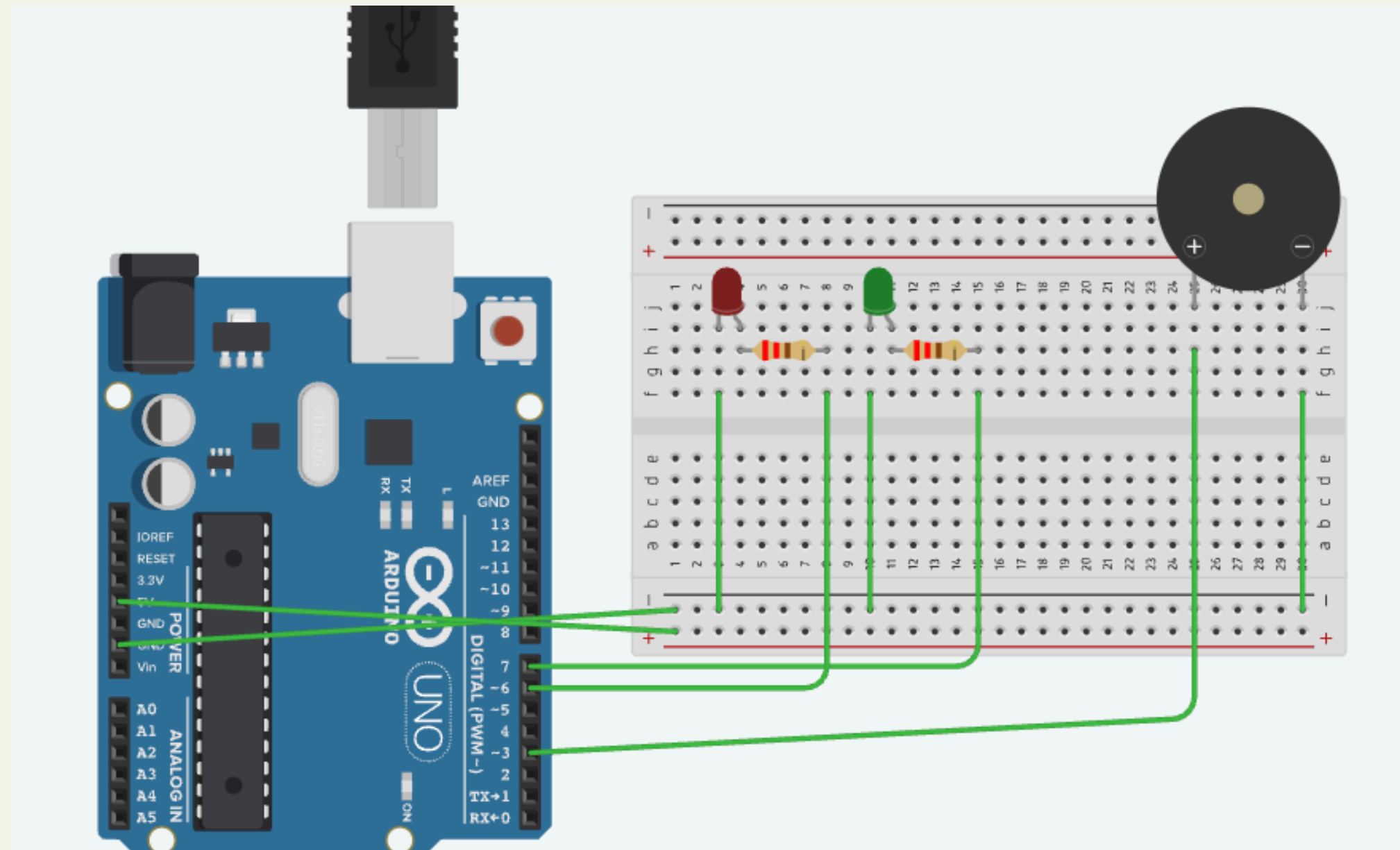
FREQUÊNCIAS DAS NOTAS MUSICAIS

- **Cada nota musical possui uma frequência específica:**
 - **Dó: 262 Hz**
 - **Ré: 294 Hz**
 - **Mi: 330 Hz**
 - **Fá: 349 Hz**
 - **Sol: 392 Hz**
 - **Lá: 440 Hz**
 - **Si: 494 Hz**
 - **#Dó: 528 Hz**
- **Programação no Arduino:**
 - **Utilização do comando `tone(pino, frequência, duração_em_ms)` para gerar frequências desejadas.**
 - **A função `noTone(pino)` desativa o buzzer.**

LABORATÓRIO 8: SEMÁFORO COM ALERTA SONORO

- **Fazer um semáforo utilizando 2 LEDs. Esse semaforo deve fazer um alerta sonoro (utilizando um buzzer), que faça um 5 bips curtos quando o sinal estiver verde, e um mais longo quando tiver amarelo (pode fazer os dois LEDs acenderem ao mesmo tempo para representar)**

LABORATÓRIO 8: SEMÁFORO COM ALERTA SONORO



LABORATÓRIO 8: SEMÁFORO COM ALERTA SONORO

```
1 // Lab 5: Semáforo com Buzzer para Deficientes Visuais
2 // Versão completa para referência do professor
3
4 // Pinos
5 int ledVermelho = 11; // Pino para LED vermelho
6 int ledVerde = 9;     // Pino para LED verde
7 int buzzer = 8;       // Pino para buzzer
8
9 void setup() {
10     // Configurar pinos como saída
11     pinMode(ledVermelho, OUTPUT);
12     pinMode(ledVerde, OUTPUT);
13     pinMode(buzzer, OUTPUT);
14 }
```

```
16 void loop() {
17     digitalWrite(ledVermelho, HIGH);
18     digitalWrite(ledVerde, LOW);
19     noTone(buzzer); // Sem som
20     delay(5000);    // Aguardar 5 segundos
21
22     digitalWrite(ledVermelho, LOW);
23     digitalWrite(ledVerde, HIGH);
24
25     for(int i = 0; i < 5; i++) {
26         tone(buzzer, 800, 100); // Bip curto de 100ms
27         delay(500);             // Aguardar 500ms entre bips
28     }
29
30     digitalWrite(ledVermelho, HIGH); // Vermelho + Verde = Amarelo
31     digitalWrite(ledVerde, HIGH);
32
33     // Som contínuo de alerta
34     tone(buzzer, 400, 2000); // Som mais grave por 2 segundos
35     delay(2000);            // Aguardar 2 segundos
36 }
```

**MUITO OBRIGADA PELA
ATENÇÃO!**

